Apache Hudi — Table Services, Incremental Queries & Streaming

Hands-on interview Q&A (companion to Databricks Interview Question Worksheet, Days 1-3)

Q1. What does Compaction do in a Merge-on-Read (MoR) Hudi table, and how do you trigger it?

Compaction merges a file group's base file with its accumulated delta log file(s) into a NEW file slice within the SAME file group (same fileId, new base instant time) — it only applies to MoR tables. Triggered via **CALL run_compaction(op => 'run', path => '<table_path>')** (Spark SQL procedure; op can be run or schedule).

Q2. Why does Time Travel (as.of.instant) still work across a Compaction or Clustering boundary, even though those operations rewrite files?

Hudi doesn't delete old file slices immediately after compaction or clustering — they're retained until the Cleaner service physically removes them per the table's retention policy (e.g. KEEP_LATEST_COMMITS). Time travel resolves "which file slice/file group was valid AS OF the given instant," so as long as the old slice is still on disk (within the Cleaner's retention window), an old as.of.instant query can still see the pre-compaction/pre-clustering state. This window is finite, not permanent.

Q3. What's the difference between Hudi's insert, upsert, bulk_insert, and delete write operations?

upsert (default) does an index lookup and merges into existing records if the key exists, else inserts. insert skips the index lookup entirely for speed — it can silently create duplicate keys if the key already exists. bulk_insert is an optimized initial-load path for large one-time loads, and also skips dedup by default. delete physically removes rows matching the given recordkey + partitionpath. Related config: hoodie.combine.before.insert (default false) controls INTRA-BATCH dedup for insert/bulk_insert only — distinct from hoodie.datasource.write.insert.drop.duplicates, which checks against EXISTING table data.

Q4. If duplicate keys already exist in a Hudi table (a broken invariant), and you run an upsert, which copy gets updated?

Not a documented, guaranteed contract — it depends on Hudi's internal Bloom Index file-processing order, an implementation detail that can change between versions. Practical lesson: never rely on upsert to self-heal existing duplicates. Run an explicit dedup job first; don't treat upsert as a fix for a pre-existing data-quality violation.

Q5. What's the real difference between Clustering and Compaction in Hudi?

Compaction is CORRECTNESS-driven, MoR-only, and merges base+log into the SAME file group (same fileId) — like updating a sticky-note change permanently into the same diary page. Clustering is PERFORMANCE-driven (bin-packing small files + sorting), works on BOTH CoW and MoR, and creates entirely NEW file groups (new fileIds), marking the old ones as "replaced" via a distinct timeline action (REPLACE, tracked as HoodieReplaceCommitMetadata) — like combining 5 separate small diaries into one brand-new sorted diary, while old diaries are retired (not deleted) until the Cleaner runs. Triggered via **CALL run_clustering(path => '<path>', order => '<col>', order_strategy => 'linear'|'z-order'|'hilbert')**.

Q6. Why can a plain Incremental Query on a Copy-on-Write Hudi table return "false positive" changed records, and how do you fix it in production?

CoW's automatic small-file packing means an unrelated, untouched record can end up sharing a rewritten file group with a record that actually changed — the incremental query then reports the whole rewritten file's records as "changed." This is architecturally inherent when small-file packing is correctly left on (as it must be in production; disabling it via hoodie.parquet.small.file.limit=0 causes a small-file explosion instead). Three layered production solutions, increasing precision/cost: (1) make the downstream consumer idempotent (MERGE/upsert instead of blind INSERT) — cheap baseline; (2) use MERGE_ON_READ, where delta log files contain only the actually-changed records, reducing false positives; (3) use Hudi's dedicated CDC feature (hoodie.table.cdc.enabled=true), which materializes exact before/after row-level change events at write time, eliminating false positives entirely.

Q7. If idempotent processing (Step 1 above) is required anyway, why bother with an "incremental" framework at all?

Idempotency and efficiency solve different problems. Idempotency makes it SAFE to reprocess an imperfect candidate set. The "incremental" framework's real value is SCALE/EFFICIENCY — scanning a small candidate set of recently-changed files instead of the entire table on every run. Same pattern as "at-least-once delivery + idempotent consumer = effectively-once processing" (Kafka consumers, CDC pipelines): the delivery mechanism doesn't need to be perfectly precise as long as the consumer is idempotent and the mechanism is efficient.

Q8. How does Hudi's Structured Streaming SOURCE (readStream.format("hudi")) track progress between micro-batches, and what happens if you need a full reload from the beginning?

HoodieStreamSource plugs into Spark's standard Structured Streaming checkpoint directory (offsets/, commits/ subfolders, one file per batch). What it stores as the "offset" is a HoodieSourceOffset — literally a Hudi commit/instant-time string, JSON-serialized — not a byte position like Kafka. Each trigger, Spark reads the last stored offset and Hudi's source converts it internally into hoodie.datasource.read.begin.instanttime for an incremental query, with the latest commit as the end. For a full reload: delete/clear the checkpointLocation and restart — with no checkpoint, Hudi falls back to an internal INIT_OFFSET ("00000000000000", older than any real commit), so the first micro-batch reads the whole table. Two safety nets: (a) Hudi's Cleaner NEVER removes the latest/current version of any record, so a full snapshot read is always complete regardless of retention aggressiveness; (b) if the incremental query's begin-instant is older than what remains on the active timeline (already moved to the archived timeline by Hudi's separate Timeline Archival service — hoodie.keep.min.commits/hoodie.keep.max.commits, a distinct mechanism from Cleaning), the config hoodie.datasource.read.incr.fallback.fulltablescan.enable (default true) makes Hudi silently fall back to a full table scan instead of erroring or silently missing data.

Q9. Hudi's dedicated CDC feature vs. a plain incremental query — what's the actual difference, and what config enables it?

A plain incremental query says "these files changed, go read the current state yourself" — on CoW tables with small-file packing this can include false positives (Q6). Hudi's CDC feature is fundamentally different: it materializes the EXACT change event at write time. Enabled via **hoodie.table.cdc.enabled = true** (default false, set only at table-creation time) plus **hoodie.table.cdc.supplemental.logging.mode** (OP_KEY_ONLY / DATA_BEFORE / DATA_BEFORE_AFTER, default DATA_BEFORE_AFTER). Read it via **hoodie.datasource.query.type=incremental** + **hoodie.datasource.query.incremental.format=cdc**. The

output has exactly 4 columns: **op** (i/u/d), **ts_ms**, **before**, **after** — a full before-image and after-image of the row (in DATA_BEFORE_AFTER mode), not just "this file changed." Verified hands-on: an update showed before(amount=100) / after(amount=150); a delete showed a real before-image with after=null; an insert showed before=null with a full after-image. Works on both COW and MOR, minimum Hudi version 0.13.0.

Q10. If the CDC output's `after` column already has the full new row, why do you need the `before` image at all?

Because looking only at `after` tells you the new state, not what actually changed. Seeing after.amount=150 alone doesn't tell you whether it moved from 100 or from 149, or whether the row is brand new. The before-image matters for audit/compliance trails ("what changed, by how much"), for downstream conflict-detection when replaying changes into another system, and for anything computing a delta rather than just a snapshot. This is exactly why Hudi's DEFAULT supplemental logging mode is the full DATA_BEFORE_AFTER, not the minimal OP_KEY_ONLY.